Week 4, Day 4: Rate Limiting, Distributed Locks, Spring Cache and Tries

Prepared for Upendra Kumar Sharma | 4 hours | Main topic: Redis rate limiting, distributed locks, Spring Cache with Redis | DSA: Tries

Today's Schedule

- **0:00 to 0:50** Study rate limiting algorithms and their Redis implementations (section 1).
- **0:50 to 1:30** Study distributed locks and their pitfalls (section 2).
- **1:30 to 2:10** Study the Spring Cache abstraction with Redis (section 3).
- **2:10 to 3:20** DSA: solve the 2 trie problems in section 5. Allow 20 minutes for the first and 40 for Word Search II.
- **3:20 to 3:50** Answer the interview questions aloud (section 6) and add 10 flashcards.
- **Optional (90 min):** do the hands-on experiments in section 4 and add rate limiting and Spring Cache to your project.

This builds on Day 3. Review cache-aside, TTL and stampede first if they feel rusty.

1. Rate Limiting with Redis

Why limit: protect services from overload and abuse, keep one noisy client from hurting others, enforce quotas and plans, and stop brute-force logins.

Where to limit: at the **API gateway** (per client, user or IP, before traffic reaches services), and inside services for expensive operations. Per-instance limiters (Resilience4j RateLimiter) only see their own traffic. To limit **across all instances**, keep the counters in a shared store such as Redis.

Algorithms

- **Fixed window counter:** count requests in each time window (for example per minute). Simple and cheap. Weakness: a client can send a full limit at the end of one window and another full limit at the start of the next, so **up to twice the limit in a short burst** crosses the boundary.
- **Sliding window log:** store a timestamp for each request (a Redis sorted set), drop entries older than the window, and count the rest. **Accurate**, but memory grows with the limit.

- **Sliding window counter:** keep counters for the current and previous window and weight the previous one by how much of it still overlaps. A cheap approximation of the sliding log.
- **Token bucket:** a bucket holds up to **capacity** tokens and refills at a steady **rate**. Each request takes a token. It allows **bursts up to the capacity** while enforcing the average rate. The most common choice for APIs.
- **Leaky bucket:** requests enter a queue and leave at a constant rate. It **smooths** traffic, and is used when the downstream needs a steady flow.

Doing it atomically: reading a counter, deciding, and then writing it in separate commands is a **race condition**, because two requests can both pass the check. Use an atomic single command (INCR) or a **Lua script**, which Redis runs as one uninterrupted step.

Fixed window (INCR plus expiry), in a script so the two steps are atomic

```
local current = redis.call('INCR', KEYS[1])
if current == 1 then
  redis.call('PEXPIRE', KEYS[1], tonumber(ARGV[1]))
end
return current
```

Sliding window log (sorted set)

```
-- KEYS[1] = key, ARGV: now(ms), window(ms), limit, unique member
local now = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
redis.call('ZREMRANGEBYSCORE', KEYS[1], 0, now - window)
if redis.call('ZCARD', KEYS[1]) < tonumber(ARGV[3]) then
  redis.call('ZADD', KEYS[1], now, ARGV[4])
  redis.call('PEXPIRE', KEYS[1], window)
  return 1
end
return 0
```

Token bucket (hash with tokens and a timestamp)

```
-- KEYS[1] = key, ARGV: rate (tokens/sec), capacity, now(ms), requested
local rate = tonumber(ARGV[1]); local capacity = tonumber(ARGV[2])
local now = tonumber(ARGV[3]); local requested = tonumber(ARGV[4])
local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(data[1]); local ts = tonumber(data[2])
if tokens == nil then tokens = capacity; ts = now end
tokens = math.min(capacity, tokens + math.max(0, now - ts) / 1000 * rate)
local allowed = 0
if tokens >= requested then tokens = tokens - requested; allowed = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(capacity / rate * 1000) * 2)
return allowed
```

Calling a script from Spring

```
@Component
public class RateLimiter {
    private final StringRedisTemplate redis;
    private final DefaultRedisScript<Long> script = new DefaultRedisScript<>
();

    public RateLimiter(StringRedisTemplate redis) {
        this.redis = redis;
        script.setLocation(new
ClassPathResource("scripts/token_bucket.lua"));
        script.setResultType(Long.class);
    }

    public boolean allow(String clientId) {
        try {
            Long allowed = redis.execute(script, List.of("rl:" + clientId),
                    "5", "10", String.valueOf(System.currentTimeMillis()),
"1");
            return allowed != null && allowed == 1L;
        } catch (RuntimeException ex) {
            return true;                        // fail open: see the design
notes below
        }
    }
}
```

Spring Cloud Gateway has this built in (a token bucket on Redis): the RequestRateLimiter filter with **replenishRate** (tokens per second), **burstCapacity** and **requestedTokens**, plus a **KeyResolver** bean that decides what to limit by.

```
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: http://order-service:8082
          predicates:
            - Path=/api/orders/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5
                redis-rate-limiter.burstCapacity: 10
                redis-rate-limiter.requestedTokens: 1
                key-resolver: "#{@userKeyResolver}"
```

Design points to say aloud

- **Choose the key:** user id, API key, IP address, or a combination. IP limits can punish many users behind one NAT.
- **Response:** HTTP **429 Too Many Requests** with a **Retry-After** header, and often X-RateLimit-Limit and X-RateLimit-Remaining headers.
- **If Redis is down: fail open or fail closed?** Failing open (allow requests) keeps the service available but removes protection. Failing closed protects the backend but blocks everyone. For most APIs choose **fail open with an alert**. For login or payment endpoints you may fail closed or use a local fallback limiter.
- **Time source:** servers' clocks can differ. Use the Redis **TIME** command inside the script, or accept small errors.
- **Cluster and cost:** one key per limiter keeps scripts in a single slot. Each check is a network round trip, so keep timeouts tight, and for very high rates add a small **local token cache** that syncs periodically.
- **Layers:** a coarse limit at the edge (per IP), a finer one per user or plan, and a protective limit on expensive endpoints.

2. Distributed Locks

First ask: do you really need a lock? Often a better tool exists.

- A **unique constraint**, a conditional update or **optimistic locking** (@Version) protects database data.
- **SELECT ... FOR UPDATE SKIP LOCKED** lets several workers share a job table safely (Week 2, Day 5).

- **Idempotency keys** make duplicate work harmless.
- **ShedLock** makes a @Scheduled job run on only one instance at a time, using a database or Redis.

A basic Redis lock

1. **Acquire:** SET lockKey token NX PX 30000. NX means only if the key does not exist, PX sets a **lease** so a crashed holder cannot block everyone forever, and the **token** is a random value that identifies the holder.
2. **Do the work.**
3. **Release safely:** delete the key **only if it still holds your token**. Do it in a Lua script so the check and delete are atomic. Without the check, a slow holder whose lease expired could delete a lock now held by someone else.

```
if redis.call('get', KEYS[1]) == ARGV[1] then
  return redis.call('del', KEYS[1])
else
  return 0
end
```

```
private static final DefaultRedisScript<Long> UNLOCK = new
DefaultRedisScript<>(
        "if redis.call('get', KEYS[1]) == ARGV[1] then return
redis.call('del', KEYS[1]) else return 0 end",
        Long.class);

public void runExclusively(String lockKey, Runnable work) {
    String token = UUID.randomUUID().toString();
    Boolean acquired = redis.opsForValue().setIfAbsent(lockKey, token,
Duration.ofSeconds(30));
    if (!Boolean.TRUE.equals(acquired)) return;              // someone else
is working
    try {
        work.run();
    } finally {
        redis.execute(UNLOCK, List.of(lockKey), token);
    }
}
```

The hard problems (this is what interviewers test)

- **The lease expires while the holder is still working** (a long GC pause, a slow query, a network stall). Two processes now believe they hold the lock. Mitigations: a lease comfortably longer than the work, **lease renewal** (a watchdog, as in the Redisson library), and above all **fencing tokens**.

- **Fencing token:** each lock acquisition gets a **monotonically increasing number**, and the protected resource (for example the database) **rejects writes carrying an older number**. This makes a late, stale holder harmless. It is the only way to make a lock safe for correctness.
- **Failover loses locks:** Redis replication is asynchronous, so after a failover a new primary may not know about a lock the old one granted. **Redlock** (acquiring on a majority of independent Redis instances) tries to address that, but it is **debated**, because it still relies on timing assumptions. A well-known critique says that for locks needed for **correctness**, you should use a consensus system (ZooKeeper, etcd) or the database with fencing.
- **Practical rule:** a Redis lock is fine when it is only an **efficiency** measure (preventing duplicate work, where occasional double execution is harmless). When a double execution would **corrupt data or money**, lock in the database or protect the resource with fencing and idempotency.
- **Libraries: Redisson** (RLock with automatic lease renewal and tryLock with wait and lease times), **Spring Integration RedisLockRegistry**, ShedLock for scheduled jobs.

3. Spring Cache with Redis

The abstraction: annotations that add caching around a method, through a **proxy**, with a pluggable store (Redis here). Enable it with **@EnableCaching**.

- **@Cacheable(cacheNames, key, condition, unless):** returns the cached value if present, otherwise runs the method and caches the result. **condition** is checked before the call, **unless** after (it can inspect #result, for example to skip null).
- **@CachePut:** always runs the method and updates the cache.
- **@CacheEvict(cacheNames, key, allEntries):** removes one entry or all entries.
- **@Caching** groups several, and **@CacheConfig** shares settings on a class.
- **Keys:** by default built from the method parameters. Use SpEL for clarity, for example key = "#sku".
- **sync = true** on @Cacheable makes concurrent callers for the same key wait for one load. It only coordinates **within one JVM**, so it reduces stampedes per instance but not across instances.

Configuration for Redis (JSON values, per-cache TTL, key prefix)

```
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
```

```
    RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        RedisCacheConfiguration base =
RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(5))
                .disableCachingNullValues()
                .prefixCacheNameWith("shop:v1:")

.serializeValuesWith(RedisSerializationContext.SerializationPair
                        .fromSerializer(new
GenericJackson2JsonRedisSerializer()));

        return RedisCacheManager.builder(factory)
                .cacheDefaults(base)
                .withCacheConfiguration("stock",
base.entryTtl(Duration.ofMinutes(1)))
                .transactionAware()          // writes and evictions happen
after the transaction commits
                .build();
    }
}
```

```
@Service
public class StockService {

    @Cacheable(cacheNames = "stock", key = "#sku", unless = "#result ==
null")
    public StockView getStock(String sku) { /* database read */ }

    @CacheEvict(cacheNames = "stock", key = "#sku")
    @Transactional
    public void setStock(String sku, int quantity) { /* database write */ }
}
```

Pitfalls and good practice

- **Proxy rules apply:** self-invocation and non-public methods bypass the cache, just like @Transactional.
- **Serialization:** the default is Java serialization, which is fragile when classes change. Use **JSON**, version your key prefix, and be careful with type information in JSON when the data may be untrusted.
- **Eviction before commit:** if the cache is cleared inside a transaction that later rolls back, or before the commit, a concurrent read can reload old data. **transactionAware()** delays cache changes until commit.

- **Cache failures must not fail requests:** by default a Redis error propagates. Provide a **CacheErrorHandler** (through CachingConfigurer) that logs and carries on, so the method simply runs without the cache.
- **What annotations do not give you:** TTL jitter, negative caching with a short TTL, cross-instance stampede protection and fine control. For critical hot paths, write the manual cache-aside code from Day 3, or wrap it behind a small service.
- **Never cache for decisions that need exact data** (stock reservation, balances).
- **Testing:** use Testcontainers with a Redis image, and assert on behavior, for example that the repository is called once for two reads.

4. Optional Hands-On (90 Minutes)

1. **Fixed window.** In redis-cli, run INCR and PEXPIRE for a key, then show the boundary burst by sending a full limit just before and just after the window resets.
2. **Sliding window and token bucket.** Load each Lua script with EVAL (or from your Spring project). Fire a quick loop of requests and print which ones are allowed.
3. **Limiter in the API.** Add the token bucket limiter to POST /api/orders per user. Send 20 requests per second against a limit of 5, and confirm 429 responses with a Retry-After header.
4. **Gateway limiter.** Configure the RequestRateLimiter route with a KeyResolver for the user or IP, and compare it with your own implementation.
5. **Failure behavior.** Stop Redis and send requests. Confirm that your chosen fail-open or fail-closed behavior happens, and that the timeouts keep latency bounded.
6. **Distributed lock.** Run two instances of the same @Scheduled job guarded by the lock above and confirm that only one runs. Then make the work last longer than the lease and watch two instances overlap. Write down why fencing or renewal is needed.
7. **Spring Cache.** Add @Cacheable to the stock read and @CacheEvict to the update. Inspect the keys with SCAN, view the JSON value and the TTL, and test the eviction. Add a CacheErrorHandler and confirm the API still works with Redis stopped.
8. **Measure.** Record latency with and without the cache, the hit ratio, and the behavior at the rate limit. Add these numbers to your README.

5. DSA: Tries (2 Problems)

What a trie is: a tree where each edge is a character, so every path from the root spells a prefix. Operations cost **O(L)** for a word of length L, regardless of how many words are stored. Uses: **autocomplete, prefix search, spell checking, IP routing tables** and word games.

Memory: an array of 26 children per node is fast but wasteful. Use a **HashMap** of children for large alphabets or sparse tries.

Problem 1: Implement Trie (Prefix Tree)

- **Idea:** each node has 26 child slots and an end-of-word flag. Insert walks and creates nodes. Search walks and checks the flag. startsWith walks and only checks that the path exists.
- **Complexity:** O(L) time per operation, and space proportional to the total characters stored.
- **Edge cases:** searching a prefix that is not a full word (false for search, true for startsWith), the empty string, repeated inserts.

```
class Trie {
    private static class Node {
        Node[] next = new Node[26];
        boolean end;
    }

    private final Node root = new Node();

    public void insert(String word) {
        Node node = root;
        for (char c : word.toCharArray()) {
            int i = c - 'a';
            if (node.next[i] == null) node.next[i] = new Node();
            node = node.next[i];
        }
        node.end = true;
    }

    public boolean search(String word) {
        Node node = find(word);
        return node != null && node.end;
    }

    public boolean startsWith(String prefix) {
        return find(prefix) != null;
    }

    private Node find(String s) {
        Node node = root;
        for (char c : s.toCharArray()) {
            node = node.next[c - 'a'];
            if (node == null) return null;
```

```
        }
        return node;
    }
}
```

Problem 2: Word Search II

- **The problem:** given a grid of letters and a list of words, return every word that can be traced through adjacent cells (up, down, left, right) without reusing a cell.
- **Idea:** searching the grid once per word is far too slow. Put **all words in a trie** and run a **DFS (backtracking) from each cell**, following the trie at the same time. If the current letter has no child in the trie, stop at once, which prunes huge parts of the search. Store the **complete word at its end node**, so you do not rebuild strings, and set it to null after finding it to avoid duplicates.
- **Visited marking:** temporarily overwrite the cell with a marker such as #, and restore it when the DFS returns.
- **Complexity:** worst case about O(M x N x 4 x 3^(L-1)) for a grid of M by N cells and a longest word of length L, but trie pruning makes it fast in practice.
- **Optimization to mention:** remove trie branches that no longer lead to unfound words.
- **Edge cases:** words longer than the grid can hold, duplicate words in the list, a single-cell grid, a word that reuses a cell (must not count).

```
class Solution {
    private static class Node {
        Node[] next = new Node[26];
        String word;
    }

    public List<String> findWords(char[][] board, String[] words) {
        Node root = new Node();
        for (String w : words) {
            Node node = root;
            for (char c : w.toCharArray()) {
                int i = c - 'a';
                if (node.next[i] == null) node.next[i] = new Node();
                node = node.next[i];
            }
            node.word = w;
        }
        List<String> result = new ArrayList<>();
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) {
                dfs(board, r, c, root, result);
```

```
            }
        }
        return result;
    }

    private void dfs(char[][] b, int r, int c, Node parent, List<String>
result) {
        char ch = b[r][c];
        if (ch == '#') return;                          // already used on
this path
        Node node = parent.next[ch - 'a'];
        if (node == null) return;                       // no word has this
prefix

        if (node.word != null) {
            result.add(node.word);
            node.word = null;                           // avoid duplicates
        }
        b[r][c] = '#';
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < b.length && nc >= 0 && nc < b[0].length) {
                dfs(b, nr, nc, node, result);
            }
        }
        b[r][c] = ch;                                   // backtrack
    }
}
```

6. Interview Questions to Answer Aloud

1. Why rate limit, and where would you do it in a microservice system?
2. Compare fixed window, sliding window, token bucket and leaky bucket. What is the boundary burst problem?
3. How would you implement a rate limiter in Redis, and why do you need a Lua script?
4. If Redis is down, should the limiter fail open or closed? Does it depend on the endpoint?
5. How do you limit across many service instances versus within one instance?
6. When do you need a distributed lock, and what are better alternatives?
7. Explain SET NX PX and why releasing needs a token check in a script.
8. What happens if a lock holder pauses longer than its lease? What is a fencing token?

9. Why is Redlock debated? When is a Redis lock acceptable and when is it not?
10. How does @Cacheable work, and why does self-invocation break it?
11. How do you configure TTL, key prefixes and JSON serialization for Spring Cache with Redis?
12. What does transactionAware do, and what problem does it prevent?
13. How do you keep a cache outage from becoming an application outage with Spring Cache?
14. What are the limits of the annotations compared with hand-written cache-aside code?

7. End-of-Day Checklist

- [ ] I can compare the five rate limiting algorithms and say why the check must be atomic.
- [ ] I can explain a Lua-based token bucket and the fail-open versus fail-closed decision.
- [ ] I can write a safe Redis lock with a token and a release script, and explain its failure modes and fencing.
- [ ] I know when to prefer database constraints, SKIP LOCKED or ShedLock over a distributed lock.
- [ ] I can configure Spring Cache with Redis, including JSON, TTL, transactionAware and an error handler.
- [ ] I solved both trie problems and can explain why a trie prunes the grid search.
- [ ] (Optional) My project has a rate limiter and cached reads, and I recorded the numbers.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 4, Day 5): project work. Wire Kafka events and the Redis cache together end to end, and write tests with JUnit, Mockito and Testcontainers.